Drone Coverage Path Planning

Lavinia Amorosi[a], Paolo Dell'Olmo[a], Justo Puerto[b], Carlos Valverde[b,*]

[a]*Department of Statistical Sciences, Sapienza University of Rome, Italy*
[b]*Department of Statistical Sciences and Operational Research, University of Seville, Spain*

Abstract

We present a second order cone mixed-integer programming formulation for a monitoring system based on one drone that moves freely in the space. We assume to have a set of not overlapping areas in the plane to be covered, represented by polygons, and one depot. The problem consists in finding the set of tours starting and ending at the depot the drone must perform in order to cover all the areas under analysis, taking into account its limited endurance. The proposed formulation embeds the drone altitude selection from a set of possible values associated with each area. This is motivated by the necessity to inspect each area with different levels of precision. Moreover, it is assumed that there is a set of points, with limited cardinality, associated with each area, from where the drone can interrupt the monitoring and exit to reach another area where the same process can be repeated. When the drone attains its minimum energy level, it must go back to the depot to swap battery and start a new tour, until all the areas have been covered. The optimization is performed by minimizing the total completion time. Preliminary computational results on a testbed of artificial instances are presented together an analysis of the obtained solutions.

Keywords: Monitoring system, Mixed-Integer Quadratic Programming, UAVs,

1. Introduction

The optimization of Unmanned aerial vehicles (UAVs) or drones operations is inherently multifaceted. It involves critical tasks such as path planning, resource allocation, energy management, and collision avoidance in increasingly crowded airspaces. For instance, determining the optimal route for a drone to cover a designated area while minimizing energy consumption and avoiding obstacles involves solving complex combinatorial and continuous optimization problems. Furthermore, when multiple drones are deployed simultaneously, issues of coordination and scheduling add an extra layer of complexity, necessitating robust algorithms capable of handling dynamic environmental conditions and regulatory constraints.

Indeed, drones have been successfully adopted in different contexts. For example, in disaster management ([1], [2]), in telecommunication for providing 5G coverage of rural zones [3], for inspecting graphs located in the plane ([4], [5], [6]). Other applications include 3D mapping and video surveillance [7], agriculture for monitoring crop growth [8], and many others (see [9] for a survey of civil use of UAVs).

*Corresponding author

In this paper we focus on coverage of areas in the plane by a single drone. We propose a mixed-integer quadratic programming (MIQP) formulation that jointly optimizes the altitude, the entry and exit points, and the sequence of operations respecting a limited energy budget. Unlike existing two-stage approaches that first plan intra-region paths and then solve an inter-region routing problem, our formulation allows the drone to interrupt the coverage of a region and resume it later, potentially reducing total energy consumption. To tackle large instances, we also develop a multi-phase constructive heuristic that provides high-quality feasible solutions in under 0.1 s and can be used to warm-start the exact solver. The main contributions of this paper are:

- A MIQP formulation for the drone coverage path planning problem integrating altitude selection, variable entry/exit points, and operation splitting under endurance constraints.
- An equivalent edge-based reformulation that replaces vertex-visitation variables with edge-coverage variables, offering a different trade-off between constraint structure and relaxation strength.
- A multi-phase heuristic combining chain selection, nearest-neighbour TSP, 2-opt improvement, greedy endurance-aware tour splitting, and coordinate-descent entry-point optimization.
- A computational study on a testbed of artificial instances with varying numbers of regions, altitude levels, and endurance values, comparing the exact solver with and without warm-start against the standalone heuristic and the two equivalent MIQP formulations.

2. State of the art

The coverage path planning problem (CPP) has been studied widely in the literature. A comprehensive survey that includes a taxonomy of CPP algorithms and drone-specific characteristics is presented in [10]. Recent work can be grouped into three broad methodological streams.

Back-and-forth coverage patterns.. Several authors adopt variants of the back-and-forth (also known as lawnmower or boustrophedon) strategy for intra-region coverage. [11] propose an improved back-and-forth algorithm with a straight-turn mode to cover rectangular regions containing forbidden areas. [12] develop a mathematical model and heuristic for energy-efficient multi-UAV multi-region CPP, using back-and-forth patterns with Bezier-curve smoothing to reduce energy consumption from turns; their optimization proceeds in two decoupled phases (intra-region path generation then inter-region routing). Similarly, [13] generate waypoints for polygonal areas and apply smoothing to reduce flight time and memory footprint.

Two-stage and sequential approaches.. Many methods first plan coverage paths within each region and then solve a separate routing problem. [14] present a CPP method for spraying drones that disinfect areas, tested on realistic scenarios. [15] propose a two-stage method for mixed regions with limited energy: they shrink perimeters to generate optimal paths, then use dynamic programming to segment the overall path among multiple drones to satisfy energy limits. [16] develop a trajectory and velocity control approach for energy-efficient visual coverage of multiple terrestrial regions, solving the problem in four sequential stages.

Energy-aware formulations with altitude and speed control.. The works most closely related to ours jointly consider energy, altitude, and speed decisions. [16] allow the UAV to change both velocity and altitude during its tour, optimizing visual coverage quality subject to energy constraints. Our approach differs from the above in that we consider the drone may interrupt coverage of a region and move to another one before returning, rather than fully covering each region in a single pass. This flexibility, enabled by the edge set $\mathcal{E}^{ext}$ in our model, can lead to shorter total tours compared to the two-stage paradigm, as illustrated in Section 3.

3. Problem Description

In this paper we study a drone coverage path planning problem. Given a set of not overlapping areas to visit, represented by polygons, one drone and one depot, the problem consists in finding the set of tours starting and ending at the depot, the drone must perform in order to cover all the areas under analysis, taking into account its limited endurance. We assume that every region can be fully covered in a single operation, that is, the drone has enough endurance to be launched from the depot, cover a region, and come back to the depot.

Let $\mathcal{R} = \{1, \ldots, R\}$ be the set of regions that must be covered by the drone. We assume that the visit of each of these regions can be performed by the drone at different altitudes following a polygonal chain. The polygonal chain associated with a region can have different shapes. Moreover, the visit of a region can be performed by the drone also with interruptions to move to another region, if this is convenient in terms of drone energy consumption. The visit of the polygonal chain associated with a region can be performed in both directions. However, in order to simplify the problem setting, we assume that for each region the number of these interruptions can be at most equal to $R - 1$. Based on that, associated with each region $r \in \mathcal{R}$, we define the set $\mathcal{I} = \{1, \ldots, 2R\}$ that represents the different points that must be located in each region, including the two endpoints of the polygonal chain. We can distinguish four sets of points:

- $\mathcal{I}_S = \{1, 2\}$, that represents the first endpoint of the polygonal chain associated with the region r. This point can be used as a launching or retrieving point.
- $\mathcal{I}_L = \{3, 5, \ldots, 2R - 1\}$, that models the launching points from region r.
- $\mathcal{I}_R = \{4, 6, \ldots, 2R\}$, that models the retrieving points from region r.
- $\mathcal{I}_T = \{2R+1, 2R+2\}$, that reports the last endpoint of the polygonal chain associated with the region r. This point can be also a launching or retrieving point.

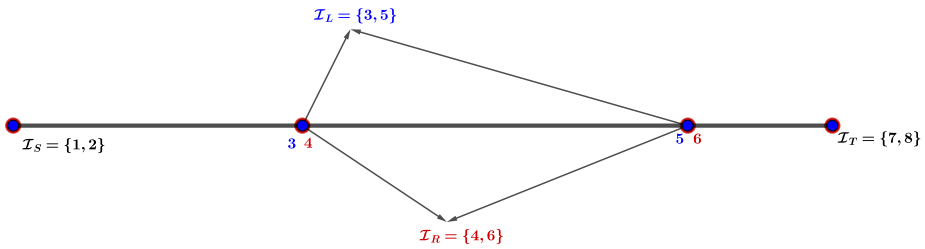

Figure 1: Set I with 3 regions

We define the graph $\mathcal{G} = (\mathcal{V} = \{v_0\} \cup \mathcal{V}', \mathcal{E} = \mathcal{E}^{int} \cup \mathcal{E}^{ext})$, induced by these sets, where:

- $v_0 = (0, 0)$ denotes the depot.
- $\mathcal{V}' = \mathcal{R} \times \mathcal{I}$ represents the set of vertices. This set can be split into $\mathcal{V}' = \mathcal{V}_S \cup \mathcal{V}_L \cup \mathcal{V}_R \cup \mathcal{V}_T$, where $\mathcal{V}_S = \mathcal{R} \times \mathcal{I}_S$, $\mathcal{V}_L = \mathcal{R} \times \mathcal{I}_L$, $\mathcal{V}_R = \mathcal{R} \times \mathcal{I}_R$, and $\mathcal{V}_T = \mathcal{R} \times \mathcal{I}_T$.
- $\mathcal{E}^{int} = \{(v, v') : v = (r, i), v' = (r, i + 1), \forall r \in \mathcal{R}, \forall i \in \mathcal{I} \setminus \{2R\}\}$ consists of the set of edges that join consecutive points in the same polygonal chain. This set can be decomposed into two subsets, named $\mathcal{E}^{int}_{LR}, \mathcal{E}^{int}_{RL}$, that represent the pairs joining consecutive pair of vertices of $\mathcal{V}_L \to \mathcal{V}_R$ ($\mathcal{V}_R \to \mathcal{V}_L$, respectively) in each region.
- $\mathcal{E}^{ext} = \{(v, v') : \forall v = (r, i) \in \mathcal{V} \setminus (\mathcal{V}_R \cup \{2, 2R+2\}), \forall v' = (r', i') \in \mathcal{V} \setminus (\mathcal{V}_L \cup \{1, 2R+1\}) : r \neq r', \forall i, i' \in \mathcal{I}\}$ consists of the set of arcs that join points from polygonal chains associated with different regions.
- $\mathcal{O}$ represents the set of operations. Each operation consists in a tour of the drone starting and ending at the depot v_0.

For each region $r \in \mathcal{R}$, we have the set $\mathcal{H}_r$ of different heights that can be used to inspect the region r. In addition, for each height $h \in \mathcal{H}_r$, we can define a set $\mathcal{P}_h$ containing different shapes of polygonal chains that ensure that the region r is covered. The shape of each polygonal chain $p \in \mathcal{P}_h$ is determined by its segments $\{[Q^s, Q^{s+1}] : s \in \mathcal{S}_p\}$, where Q^s and Q^{s+1} denote the end points of the segment s and $\mathcal{S}_p = \{1, \ldots, S_p\}$, the set of segments associated with the polygonal chain p.

We assume that air density, denoted by ρ, depends on height as stated in the following function (derived from the ideal gas law):

$$\rho_h = \frac{p_0 M}{C_R T_0} \left(1 - \frac{Lh}{T_0}\right)^{\frac{gM}{C_R L} - 1},$$

where the constant can be found in the International Standard Atmosphere (ISA) model:

- $p_0 = 101325$ Pa: sea level standard atmospheric pressure.
- $T_0 = 288.15$ K: sea level standard temperature.
- $g = 9.80665\ m/s^2$: earth-surface gravitational acceleration.
- $L = 0.0065\ K/m$: temperature lapse rate.
- $C_R = 8.31446\ J/(mol \cdot K)$: universal gas constant.
- $M = 0.0289652\ kg/mol$: molar mass of dry air.

In addition, we assume that the wind has a constant direction represented by the unitary vector $\vec{u}^w$. The module of the wind speed depends on the height following the formula [17]:

$$\nu_h^w = \nu_{10}^w \left(\frac{h}{10}\right)^a,$$

where ν_{10}^w represents the wind speed at height 10 meters and a is the Hellmann exponent, that depends upon the coastal location, the shape of the terrain on the ground, and the stability of the air. Hence, the vector of the wind speed is given by $\vec{\nu}_h^w = \nu_h^w \vec{u}^w$.

Once we have defined the speed of the wind for each altitude, we can compute the speed that the drone must follow, $\vec{\nu}_h^d$, to reach a specified constant speed $\vec{\nu}$, that is

$$\vec{\nu}_h^d = \vec{\nu} - \vec{\nu}_h^w.$$

If the drone moves from one region r to another region r', we compute this speed ($\vec{\nu}_{rr'h}^d$) by taking the direction joining the center of both regions to avoid bilinear terms in the formulation. However, since the polygonal chain that covers the region is fixed a priori (and hence, the direction of its segments), the drone speed can be also computed beforehand.

One key feature of our problem setting is that the drone is not forced to fully explore one region before moving to another one. In contrast, the set of edges $\mathcal{E}^{ext}$ allows the drone to leave the region not fully covered to explore a different one.

Other contributions for the CPP propose a two-stage approach, that is, the optimization procedure fully explores each region and then inter region routing problem is solved.

It can be seen with some examples that the flying pattern proposed in this article can reduce the length of the tour as opposed to the two-stage approach.

For this purpose, we analyze a specific problem instance showing the difference of the tour lengths of these two approaches.

Suppose to have an instance composed by one large region which is a regular polygon of n edges of length l and n "unit size" (or drone visibility radius size) small satellite regions external to the regular polygon, each one at a very small distance, let us assume unit distance (or radius distance) for simplicity, from the vertices of the regular polygon. See Figure 2 for an example with $n = 3$. Let us assume, again for simplicity, that the two approaches require the same path length to cover the regular polygon. The path to cover the satellite regions with our approach would require to leave the polygon in correspondence to each vertex to

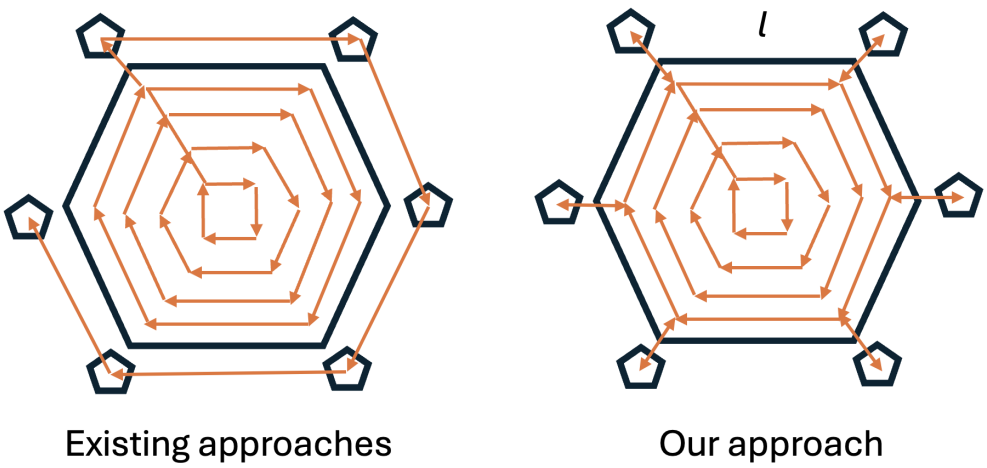

Figure 2: Comparison of two-stage and the proposed approach

cover the satellite region closest to that vertex. The cost of the complete tour would then be the cost to cover the regular polygon plus $n * 4$, as me assumed that satellite regions are at distance 1 from the vertices and the flying pattern is one unit internal to the perimeter of the polygon and the drone movement is from the polygon to the satellite region and back. In the two stage approach the cost of the complete tour would be, in the best case, the cost to cover the polygon plus all the movements from one satellite vertex to the closest one. Hence, in the two stage case to cover all the regions the tour length would be the one to cover the polygon plus $(n-1)l$. It is easy to see that the ratio of the difference between the length of this tour with the one obtained with the proposed approach is $(n-1)l/4*n$, and hence can be arbitrarily large for large values of l.

Indeed, it can be easily shown that the worst case for the two-stage approach concerning this ratio is obtained when the regular polygon is a triangle and the ratio of the difference of tour lengths is $l/6$.

It has to be noted that our approach does not force the drone path to leave region not completely covered to move to a different one, but it allows the drone to do this whenever it minimizes the objective function.

4. Mathematical Formulation

This section presents a mixed-integer quadratic programming formulation for the problem outlined in Section 3. First, we define the constraints that determine the placement of points along the polygonal chain covering each polygon. Next, we establish the constraints governing the sequence of operations performed by the drone. Then, we introduce the distance constraints that model the drone movements. Finally, we specify the constraints related to energy consumption.

We then define the decision variables representing the problem, which are summarized in Table 1 and Table 2.

4.1. Location constraints

To set the coordinates for each point P_v of the graph, we define two family of variables. The first family of binary variables, μ_v^s, decides which segment s of the polygonal chain

Table 1: Corrected Table of Variables

Name	Domain	Range	Description
μ_v^s	$\mathcal{V}_2 \times \{s \in \mathcal{S}_t : t \in \mathcal{T}_r\}$	$\{0,1\}$	1, if vertex $v = (r,i)$ is located at the segment $s \in \mathcal{S}_t$, where $t \in \mathcal{T}_r$.
α_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$\{0,1\}$	1, if polygonal chain t is selected.
$x_{vv'}^o$	$\mathcal{E} \times \mathcal{O}$	$\{0,1\}$	1, if the drone goes from vertex v to vertex v' in operation o.
y_v^o	$\mathcal{V}' \times \mathcal{O}$	$\{0,1\}$	1, if vertex v is visited during operation o.
ζ^o	$\mathcal{O}$	$\{0,1\}$	1, if all vertices are visited before starting operation o.
k^o	$\mathcal{O}$	$[1,2,\ldots,\lvert\mathcal{V}'\rvert]$	number of vertices visited in operation o.
$\varphi_{rr'}$	$\{(r,r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0,1\}$	1, if the drone moves from region r to region r' at the height selected to visit region r.
$\psi_{rr'}$	$\{(r,r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0,1\}$	1, if the height selected to visit region r is lower than the height selected to visit region r'.

Table 2: Table of Variables

Name	Domain	Range	Description
P_v	$\mathcal{V}'$	$\mathbb{R}^3$	Coordinates of the point associated to vertex v.
γ_v	$\mathcal{V}'$	$[0,1]$	Parameterization of vertex v in the segment selected to locate it.
λ_v	$\mathcal{V}'$	$[0,S_t] : t \in \mathcal{T}_r$	Global parameterization of vertex v in the polygonal chain selected to visit the region r.
$d_{vv'}^t$	$\mathcal{E}^{int} \times \mathcal{T}_r$	$\mathbb{R}^+$	Distance between vertices inside the polygonal chain t selected to visit the region r.
$d_{vv'}$	$\mathcal{E}$	$\mathbb{R}^+$	Distance between vertex v and vertex v'.
$d_{vv'}^{xy}$	$\mathcal{E}^{ext}$	$\mathbb{R}^+$	Horizontal distance between vertex v and vertex v'.
$d_{vv'}^{z}$	$\mathcal{E}^{ext}$	$\mathbb{R}^+$	Vertical distance between vertex v and vertex v'.
$w_{vv'}^t$	$\mathcal{E}^{int} \times \mathcal{T}_r$	$\mathbb{R}^+$	Drone energy consumption between vertices inside the polygonal chain t selected to visit the region r.
$w_{vv'}$	$\mathcal{E}$	$\mathbb{R}^+$	Drone energy consumption between vertex v and vertex v'.

is selected for the vertex v. Once a segment s is chosen, the second family of continuous variables, $\gamma_v \in [0,1]$, represents P_v as a linear combination of the endpoints of the segment s. Putting all together, the following constraints determine the location of P_v:

$$P_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} \mu_v^s \left[(1-\gamma_v) Q^s + \gamma_v Q^{s+1} \right], \qquad \forall v = (r,i) \in \mathcal{V}', \qquad (Loc - C_1)$$

$$\sum_{s \in \mathcal{S}_t} \mu_v^s = \alpha_t, \qquad \forall v = (r,i) \in \mathcal{V}', \forall t \in \mathcal{T}_r, \qquad (Loc - C_2)$$

$$\sum_{t \in \mathcal{T}_r} \alpha_t = 1, \qquad \forall r \in \mathcal{R}. \qquad (Loc - C_3)$$

Constraints $(Loc - C_2)$ describe the binary variable α_p, that attains value one if we select the polygonal shape p for the region r. Constraints $(Loc - C_3)$ ensure that a polygonal shape must be selected for each region r.

Once points are represented by means of variables μ and γ, we can order these points by defining a continuous variable $\lambda_v \in [0, S_p]$ that gives the global parameterisation of the vertex v in the polygonal chain p assigned to region r. The following constraints set the values of these variables:

$$\lambda_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} (s-1)\mu_v^s + \gamma_v, \qquad \forall v = (r,i) \in \mathcal{V}', \qquad (Loc - C_4)$$

$$\lambda_v = 0, \qquad \forall v = (r,1), (r,2) : r \in \mathcal{R}, \qquad (Loc - C_5)$$

$$\lambda_v \leq \lambda_{v'}, \qquad \forall v, v' \in \mathcal{V}' : (v,v') \in \mathcal{E}_{RL}^{int} \cup \mathcal{E}_{RL}^{S} \cup \mathcal{E}_{RL}^{T}, \qquad (Loc - C_6)$$

$$\lambda_v = \lambda_{v'}, \qquad \forall v, v' \in \mathcal{V}' : (v,v') \in \mathcal{E}_{LR}^{int} \cup \mathcal{E}_{LR}^{S} \cup \mathcal{E}_{LR}^{T}, \qquad (Loc - C_7)$$

$$\lambda_v = \sum_{t \in \mathcal{T}_r} \alpha_t S_t, \qquad \forall v = (r, 2R+1), (r, 2R+2) : r \in \mathcal{R}. \qquad (Loc - C_8)$$

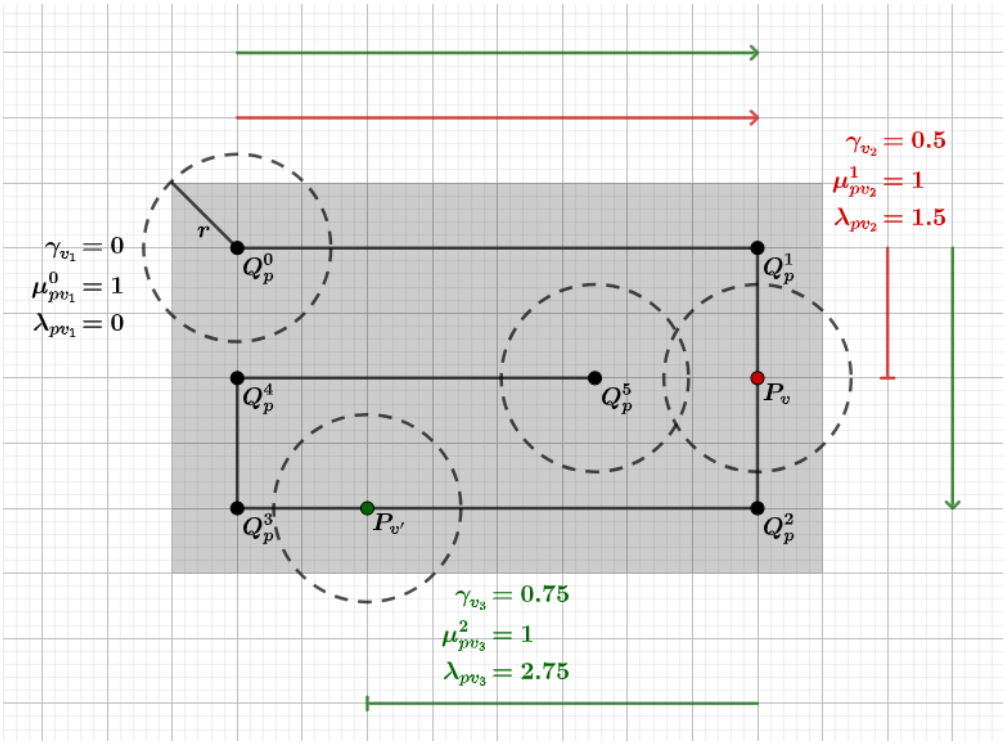

Figure 3: Parameterization of vertices

Constraints $(Loc - C_4)$ define the values of λ_v in relation with the selected segment s and the fraction of this segment that is traversed. Constraints $(Loc - C_5)$ ensure that the starting points of the polygonal chain correspond to the vertices $(r, 1)$ and $(r, 2)$. Constraints $(Loc - C_6)$ ensure that the sequence of points located on the polygonal chain respects the ordering imposed by the direction of traversal, i.e. a vertex assigned to a later segment must have a larger parameter λ than one assigned to an earlier segment. Constraints $(Loc - C_7)$ enforce that consecutive pairs (launch–retrieve points) share the same parameter λ, meaning they coincide in space and the drone can enter and exit the chain at the same position. Finally, constraints $(Loc - C_8)$ fix λ to the total length S_t of the selected chain t for the end vertices $(r, 2R + 1)$ and $(r, 2R + 2)$, ensuring the polygonal chain is fully traversed.

4.2. Drone path constraints

In this subsection, we describe the set of constraints related to the operations that the drone must perform to visit all vertices described in $\mathcal{V}'$. In this context, we define four families of variables that model the path. Let $x^o_{vv'}$ be a binary variable that is one if the drone goes from vertex v to vertex v' in operation o. Let y^o_v be a binary variable that is one if the vertex v is visited during the operation o. Let also ζ^o be a binary variable that is one if all vertices are visited before starting the operation o. Finally, we count the number of vertices visited in operation o by means of the integer variable k^o.

$$\sum_{v' \in \mathcal{V}} x^o_{v_0 v'} = 1 - \zeta^o, \qquad \forall o \in \mathcal{O}, \qquad (D_{path} - C_1)$$

$$\sum_{v' \in \mathcal{V}} x^o_{v'v} - \sum_{v' \in \mathcal{V}} x^o_{vv'} = 0, \qquad \forall v \in \mathcal{V}', \forall o \in \mathcal{O}, \qquad (D_{path} - C_2)$$

$$\sum_{v' \in \mathcal{V}} x^o_{v'v_0} = 1 - \zeta^o, \qquad \forall o \in \mathcal{O}, \qquad (D_{path} - C_3)$$

$$\sum_{v' \in \mathcal{V}} x^o_{vv'} = y^o_v, \qquad \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, \qquad (D_{path} - C_4)$$

$$\sum_{v' \in \mathcal{V}} x^o_{v'v} = y^o_v, \qquad \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, \qquad (D_{path} - C_5)$$

$$\sum_{o \in \mathcal{O}} y^o_v = 1, \qquad \forall v \in \mathcal{V}', \qquad (D_{path} - C_6)$$

$$\sum_{v,v' \in \mathcal{V}''} x^o_{vv'} \leq |\mathcal{V}''| - 1, \qquad \forall \mathcal{V}'' \subseteq \mathcal{V}, \forall o \in \mathcal{O}, \qquad (D_{path} - C_7)$$

$$\sum_{o \in \mathcal{O}} x^o_{vv'} = 1, \qquad \forall v, v' \in \mathcal{V}' : (v, v') \in \mathcal{E}^{int}_{RL} \cup \mathcal{E}^{S}_{RL} \cup \mathcal{E}^{T}_{RL}. \qquad (D_{path} - C_8)$$

Constraints $(D_{path} - C_1)$ ensure that the drone exits from the depot if there is a vertex that is not already visited in operation o. Constraints $(D_{path} - C_2)$ are the flow conservation constraints. Constraints $(D_{path} - C_3)$ guarantee that drone operation ends in the depot if there is a vertex that has not been visited in operation o. Constraints $(D_{path} - C_4)$ and $(D_{path} - C_5)$ ensure that the vertex v is visited if there is a path passing through this vertex. Constraints $(D_{path} - C_6)$ impose that every vertex must be visited in some operation. Inequalities $(D_{path} - C_7)$ represent the subtour elimination constraints for each tour made in each operation. Finally, constraints $(D_{path} - C_8)$ guarantee that each polygonal chain is completed traversed. They impose that all the edges of the path graph associated with each region are traversed in some operation.

Valid Inequalities

In this subsection, we present a set of valid inequalities for the representation of the drone path that are included in the formulation to enhance it. Let ζ^o be a binary variable that takes the value of one if all vertices have been visited before starting the operation o, and zero otherwise. If this is the case, then all vertices must also be visited before the operation $o + 1$. Therefore, the ζ variables must satisfy the following monotonicity constraint:

$$\zeta^o \leq \zeta^{o+1}, \quad \forall o \in \mathcal{O} \setminus \{O\}. \qquad \text{(Monotonicity)}$$

We define k^o as the number of vertices visited during the operation o, excluding the depot. The variables y can be used to determine this quantity, where we recall that y^o_v equals one if the vertex v is visited in operation o. Thus:

$$k^o = \sum_{v \in \mathcal{V}'} y^o_v, \quad \forall o \in \mathcal{O}. \qquad (k - C)$$

If ζ^o is equal to one, it implies that all vertices in $\mathcal{V}'$ must have been visited in previous operations. This relationship is enforced through the following inequality:

$$\sum_{o'=1}^{o-1} k^{o'} \geq |\mathcal{V}'|\zeta^o, \quad \forall o \in \mathcal{O}. \tag{$VI - C_1$}$$

To refine the feasible solution space, we assume, without loss of generality, that an operation o cannot consists in remaining at the depot, if there are still targets left to visit. This requirement is ensured by the following constraint:

$$k^o \geq 1 - \zeta^o, \quad \forall o \in \mathcal{O}. \tag{$VI - C_2$}$$

4.3. Distance constraints

This subsection is devoted to computing the distance between any pair of vertices of the graph. The reader may note that these distances must be modelled using continuous variables, since the location of the points is not fixed in the polygonal chains representing the regions.

Firstly, we compute the distance between two points that belong to the same polygonal chain. For every $v, v' \in \mathcal{V}'$ such that $(v, v') \in \mathcal{E}^{int}_{RL}$, the distance inside the polygonal chain is represented by the following expressions:

$$d^{ss'}_{vv'} = \begin{cases} (\gamma_{v'} - \gamma_v)L(s), & \text{if } P_v, P_{v'} \in [Q^s, Q^{s+1}], \\ (1-\gamma_v)L(s) + \sum\limits_{s''=s+1}^{s'-1} L(s'') + \gamma_{v'}L(s'), & \text{if } P_v \in [Q^s, Q^{s+1}] \text{ and } P_{v'} \in [Q^{s'}, Q^{s'+1}], \end{cases}$$

where $L(s) = \|Q^{s+1} - Q^s\|_2$ denotes the length of the segment s. The reader may note that for the cases where $v >_{lex} v'$, we can set $d^t_{vv'} = d^t_{v'v}$ to obtain the same value for the symmetric case. The expression above can be rewritten by means of μ variables as follows:

$$d^{ss'}_{vv'} = \begin{cases} \mu^s_v \mu^s_{v'} \left[(\gamma_{v'} - \gamma_v)L(s)\right] + \mu^s_v \mu^{s'}_{v'} \left[(1-\gamma_v)L(s) + \sum\limits_{s''=s+1}^{s'-1} L(s'') + \gamma_{v'}L(s')\right], & \text{if } v <_{lex} v', \\ d^{ss'}_{v'v}, & \text{if } v >_{lex} v'. \end{cases} \tag{$Dist - C_1$}$$

Hence, since each vertex is only assigned to a single segment by $(Loc - C_2)$ and $(Loc - C_3)$, the distance between v and v' is given by:

$$d_{vv'} = \sum_{t \in \mathcal{T}_r} \sum_{s, s' \in \mathcal{S}_t} d^{ss'}_{vv'}, \quad \forall v = (r, i), v' = (r, i') \in \mathcal{V}' : (v, v') \in \mathcal{E}^{int}_{RL}. \tag{$Dist - C_2$}$$

Secondly, we compute the distance between vertices of different regions. In that case, we assume that drone movement is decomposed into two parts: horizontal movement in the plane OXY and vertical movement on the axis OZ. We denote by $d^{xy}_{vv'}$ and $d^z_{vv'}$ the continuous variables that represent the horizontal and vertical distances travelled by the drone to go from P_v to $P_{v'}$, respectively. Thus, the constraints that compute the total distance are the following:

$$d_{vv'} = d^{xy}_{vv'} + d^z_{vv'}, \qquad \forall v, v' \in \mathcal{V} : (v, v') \in \mathcal{E}^{ext}, \tag{$Dist - C_3$}$$
$$d^{xy}_{vv'} \geq \|(P^x_v, P^y_v, 0) - (P^x_{v'}, P^y_{v'}, 0)\|_2, \qquad \forall v, v' \in \mathcal{V} : (v, v') \in \mathcal{E}^{ext}, \tag{$Dist - C_4$}$$
$$d^z_{vv'} \geq P^z_v - P^z_{v'}, \qquad \forall v, v' \in \mathcal{V} : (v, v') \in \mathcal{E}^{ext}, \tag{$Dist - C_5$}$$

$$d^z_{vv'} \geq P^z_{v'} - P^z_v, \qquad \forall v, v' \in \mathcal{V} : (v, v') \in \mathcal{E}^{ext}. \qquad (Dist - C_6)$$

Here, we denote the coordinates of P_v as $P_v = (P^x_v, P^y_v, P^z_v)$. Constraint $(Dist - C_3)$ computes the total distance of the horizontal and vertical components. The horizontal distance, d^{xy}, is computed as the Euclidean distance by means of second-order cone constraints $(Dist - C_4)$. The vertical distance, d^z, is calculated by taken the absolute value of the difference of the third components that is linearised by using the constraints $(Dist - C_5)$ and $(Dist - C_6)$.

4.4. Consumption constraints

In our model, we assume that the drone has a limited endurance, denoted by $W^{\max}$, to perform each operation. The consumption is computed by using the drag force formula, whose general expression is:

$$w = \frac{1}{2} A c \rho \nu d,$$

where A and c denote the front surface and the drag coefficient of the drone, respectively; ρ represents the air density; ν, the speed of the drone (dependent on the height), and d, the distance travelled. Similarly to what has been done for expressing the distance between vertices, we first distinguish the case in which they belong to the same region to the one in which they belong to different regions.

4.4.1. Consumption between vertices of the same region

If the two vertices belong to the same region we can express the drone energy consumption as follows:

If $v <_{lex} v'$

$$w^t_{vv'} = \begin{cases} E^{xy} \rho_s |\vec{\nu}^d_s| (\gamma_{v'} - \gamma_v) L(s), & \text{if } P_v, P_{v'} \in [Q^s, Q^{s+1}], \quad s \in \mathcal{S}_t, \\ E^{xy} \rho_s |\vec{\nu}^d_s| (1 - \gamma_v) L(s) + \sum\limits_{s''=s+1}^{s'-1} E^{xy} \rho_{s''} |\vec{\nu}^d_{s''}| L(s'') + E^{xy} \rho_{s'} |\vec{\nu}^d_{s'}| \gamma_{v'} L(s'), & \text{if } P_v \in [Q^s, Q^{s+1}] \text{ and } P_{v'} \in [Q^{s'}, Q^{s'+1}], \quad s, s' \in \mathcal{S}_t : s \neq s' \end{cases}$$

If $v >_{lex} v'$

$$w^t_{vv'} = \begin{cases} E^{xy}\rho_s|\vec{\nu}^d_s|(\gamma_v - \gamma_{v'})L(s), & \text{if } P_v, P_{v'} \in [Q^s, Q^{s+1}], \quad s \in \mathcal{S}_t, \\ E^{xy}\rho_s|\vec{\nu}^d_s|\gamma_v L(s) + \sum_{s''=s+1}^{s'-1} E^{xy}\rho_{s''}|\vec{\nu}^d_{s''}|L(s'') + E^{xy}\rho_{s'}|\vec{\nu}^d_{s'}|(1-\gamma_{v'})L(s'), & \text{if } P_v \in [Q^s, Q^{s+1}] \text{ and } P_{v'} \in [Q^{s'}, Q^{s'+1}], \quad s, s' \in \mathcal{S}_t : s \neq s' \end{cases}$$

where $E^{xy} = \frac{1}{2}cA$ and $\vec{\nu}^d_s$ is the drone speed to traverse segment s of the polygonal chain. This speed is computed beforehand as:

$$\vec{\nu}^d_s = \begin{cases} \nu_0 \frac{Q^{s+1}-Q^s}{\|Q^{s+1}-Q^s\|} - \vec{\nu^w_h}, & \text{if } v <_{lex} v', \\ \nu_0 \frac{Q^s-Q^{s+1}}{\|Q^s-Q^{s+1}\|} - \vec{\nu^w_h}, & \text{if } v >_{lex} v'. \end{cases}$$

The expression above can be rewritten by means of the μ variables:

$$w^t_{vv'} = \begin{cases} \mu^s_v \mu^s_{v'} E^{xy}\rho_s|\vec{\nu}^d_s|(\gamma_{v'} - \gamma_v)L(s) + \mu^s_v \mu^{s'}_{v'} \left[E^{xy}\rho_s|\vec{\nu}^d_s|(1-\gamma_v)L(s) + \sum_{s''=s+1}^{s'-1} E^{xy}\rho_{s''}|\vec{\nu}^d_{s''}|L(s'') + E^{xy}\rho_{s'}|\vec{\nu}^d_{s'}|\gamma_{v'}L(s') \right], & s \in \mathcal{S}_t \quad \text{if } v <_{lex} v' \\ \mu^s_v \mu^s_{v'} E^{xy}\rho_s|\vec{\nu}^d_s|(\gamma_v - \gamma_{v'})L(s) + \mu^s_v \mu^{s'}_{v'} \left[E^{xy}\rho_s|\vec{\nu}^d_s|\gamma_v L(s) + \sum_{s''=s+1}^{s'-1} E^{xy}\rho_{s''}|\vec{\nu}^d_{s''}|L(s'') + E^{xy}\rho_{s'}|\vec{\nu}^d_{s'}|(1-\gamma_{v'})L(s') \right], & s, s' \in \mathcal{S}_t : s \neq s' \quad \text{if } v >_{lex} v' \end{cases} \quad (w - C_1)$$

Since each vertex is only assigned to a single segment, the drone energy consumption between vertex v and vertex v' is given by:

$$w_{vv'} = \sum_{t \in \mathcal{T}_r} \sum_{s,s' \in \mathcal{S}_t} w^t_{vv'}, \quad \forall v = (r,i), v' = (r,i') \in \mathcal{V}' : (v,v') \in \mathcal{E}^{int}. \quad (w - C_2)$$

4.4.2. Consumption between vertices from different regions

If the two vertices belong to different regions, we need to introduce the binary variable $\varphi_{rr'}$ that takes the value of one if the height selected to move the drone from region r to region $r^{'}$ is the one adopted to visit region r. Based on that, the energy consumption also depends on the relation between the height selected to visit region r (denoted by h_r) and the one selected to visit region $r^{'}$ (denoted by $h_{r'}$). For this reason we need also to define the binary variable $\psi_{rr'}$ that takes value one if $h_r < h_{r'}$. Thus, the drone energy consumption can be expressed as follows:

$$\begin{aligned} w_{vv'} =& \varphi_{rr'} \left(E^{xy} d^{xy}_{vv'} \sum_{t \in \mathcal{T}_r} \alpha_t \rho_t |\vec{\nu}^d_{rr't}| + E^z \psi_{rr'} d^z_{vv'} \frac{1}{2} \left(\sum_{t \in \mathcal{T}_r} \alpha_t \rho_t + \sum_{t' \in \mathcal{T}_{r'}} \alpha_{t'} \rho_{t'} \right) \nu^z \right) + \quad (w - C_3) \\ & + (1 - \varphi_{rr'}) \left(E^{xy} d^{xy}_{vv'} \sum_{t' \in \mathcal{T}_{r'}} \alpha_t \rho_t |\vec{\nu}^d_{rr't'}| + E^z (1 - \psi_{rr'}) d^z_{vv'} \frac{1}{2} \left(\sum_{t \in \mathcal{T}_r} \alpha_t \rho_t + \sum_{t' \in \mathcal{T}_{r'}} \alpha_{t'} \rho_{t'} \right) \nu^z \right) \end{aligned}$$

where $E^{xy} = \frac{1}{2}cA$, $E^z = \frac{1}{2}cA$, are constants representing the characteristics of the drone, and ν^z defines the module of the vertical speed of the drone.

Based on the previous expressions, we can introduce the last set of constraints, imposing that the drone energy consumption associated with each operation o is not greater than the its maximum endurance $W^{\max}$:

$$\sum_{(v,v')\in\mathcal{E}} w_{vv'} x^o_{vv'} \leq W^{\max} \quad \forall o \in \mathcal{O} \qquad (Endurance)$$

4.5. Formulation

The complete formulation of the CPP follows:

$$\begin{aligned} minimize \quad & W = \sum_{v,v'\in\mathcal{V}} \sum_{o\in\mathcal{O}} d_{vv'} x^o_{vv'} && \text{(Formulation)} \\ \text{subject to} \quad & (D_{path} - C_1) - (D_{path} - C_7), \\ & (\text{Monotonicity}), (k - C), (VI - C_1) - (VI - C_2), \\ & (Loc - C_1) - (Loc - C_7), \\ & (Dist - C_1) - (Dist - C_6), \\ & (w - C_1) - (w - C_3), \\ & (Endurance) \end{aligned}$$

4.6. Alternative edge-based formulation

Table 3: Binary and integer variables for the edge-based formulation

Name	Domain	Range	Description
μ^s_v	$\mathcal{V}' \times \{s \in \mathcal{S}_t : t \in \mathcal{T}_r\}$	$\{0,1\}$	1, if vertex $v = (r,i)$ is located at the segment $s \in \mathcal{S}_t$, where $t \in \mathcal{T}_r$.
α_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$\{0,1\}$	1, if polygonal chain t is selected.
$x^o_{vv'}$	$\mathcal{E} \times \mathcal{O}$	$\{0,1\}$	1, if the drone traverses the edge (v,v') in operation o.
y^o_{uv}	$\mathcal{E}^{int} \times \mathcal{O}$	$\{0,1\}$	1, if the intra-ring edge (u,v) is covered in operation o.
ζ^o	$\mathcal{O}$	$\{0,1\}$	1, if all vertices are visited before starting operation o.
k^o	$\mathcal{O}$	$[1,2,\ldots,\lvert\mathcal{E}^{int}\rvert]$	number of intra-ring edges covered in operation o.
$\varphi_{rr'}$	$\{(r,r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0,1\}$	1, if the drone moves from region r to region r' at the height selected to visit region r.
$\psi_{rr'}$	$\{(r,r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0,1\}$	1, if the height selected to visit region r is lower than the height selected to visit region r'.

The formulation presented above treats each vertex in $\mathcal{V}'$ individually: location variables μ^s_v, γ_v are defined per vertex, and path variables $x^o_{vv'}$ are defined per pair of vertices. When the number of regions grows, the combinatorial explosion of vertex pairs—especially in $\mathcal{E}^{ext}$—can make the model difficult to solve. An alternative representation can be obtained by replacing the per-vertex visitation variables y^o_v with per-edge coverage variables y^o_{uv} defined on the set of intra-ring edges $\mathcal{E}^{int}$, following a perspective akin to the rural postman problem.

In this alternative formulation, the location constraints (Loc-C_1 through Loc-C_5, ρ-selection) and the distance and energy constraints (Dist-C, w-C, Endurance) remain unchanged, as they do not depend on y^o_v. The binary and integer variables for this formulation are listed in Table 3 (the continuous variables are the same as in Table 2 of the vertex-based formulation). The drone path constraints are modified as follows.

Drone path constraints (edge-based).. The flow conservation (D_{path}-C_2) and MTZ subtour elimination (D_{path}-C_7) are kept unchanged. The depot departure and return constraints (D_{path}-C_1, D_{path}-C_3) also remain the same. The degree constraints, however, are relaxed: instead of coupling each traversal to a vertex-visitation variable, we simply bound the number of traversals incident to each vertex:

$$\sum_{v' \in \mathcal{V}} x^o_{vv'} \leq 1, \qquad \forall v \in \mathcal{V},\ \forall o \in \mathcal{O}, \qquad (\mathrm{D}'_{\mathrm{path}}\text{-}\mathrm{C}_4)$$

$$\sum_{v' \in \mathcal{V}} x^o_{v'v} \leq 1, \qquad \forall v \in \mathcal{V},\ \forall o \in \mathcal{O}, \qquad (\mathrm{D}'_{\mathrm{path}}\text{-}\mathrm{C}_5)$$

Constraint ($\mathrm{D}'_{\mathrm{path}}$-$\mathrm{C}_4$) (resp. ($\mathrm{D}'_{\mathrm{path}}$-$\mathrm{C}_5$)) ensures that at most one edge leaves (resp. enters) each vertex in a given operation, which together with flow conservation guarantees that each vertex is visited at most once per operation. Since the set $\mathcal{E}^{int}$ is directed (each intra-ring edge is traversed only in the forward direction), the constraint $\mathrm{D}_{\mathrm{path}}$-$\mathrm{C}_8$ (each intra edge traversed exactly once) remains unchanged.

Edge-coverage linking constraints.. For each intra-ring edge $(u, v) \in \mathcal{E}^{int}$ we introduce a binary variable y^o_{uv} that takes value 1 if the edge is covered (i.e. traversed in either direction) during operation o. The following constraints link y^o_{uv} with the traversal variables x^o_{uv}:

$$\sum_{o \in \mathcal{O}} y^o_{uv} = 1, \qquad \forall (u, v) \in \mathcal{E}^{int}, \qquad (\mathrm{EC}_1)$$

$$y^o_{uv} \geq x^o_{uv}, \qquad \forall (u, v) \in \mathcal{E}^{int},\ \forall o \in \mathcal{O}, \qquad (\mathrm{EC}_{2a})$$

$$y^o_{uv} \leq x^o_{uv}, \qquad \forall (u, v) \in \mathcal{E}^{int},\ \forall o \in \mathcal{O}. \qquad (\mathrm{EC}_3)$$

Constraint (EC_1) ensures that each required edge is assigned to exactly one operation. Constraints (EC_{2a}) and (EC_3) together enforce $y^o_{uv} = x^o_{uv}$—the edge is covered if and only if it is traversed (recall that intra-ring edges are directed, so the reverse direction x^o_{vu} does not exist). The edge-coverage variables y^o_{uv} replace the vertex-visitation variables y^o_v in the valid inequalities.

Adapted valid inequalities.. The monotonicity constraint (Monotonicity) and VI-2 ($VI - C_2$) are unchanged. The k-counting constraint and VI-1 are updated to use the edge-based variables:

$$k^o = \sum_{(u,v) \in \mathcal{E}^{int}} y^o_{uv}, \qquad \forall o \in \mathcal{O}, \qquad (k\text{-}\mathrm{C}')$$

$$\sum_{o'=1}^{o-1} k^{o'} \geq |\mathcal{E}^{int}|\, \zeta^o, \qquad \forall o \in \mathcal{O}. \qquad (\text{VI-}1')$$

Equivalence.. Because constraints (EC_{2a}) and (EC_3) give $y^o_{uv} = x^o_{uv}$, and $\mathrm{D}_{\mathrm{path}}$-$\mathrm{C}_8$ forces $\sum_o x^o_{uv} = 1$, we have $y^o_{uv} = x^o_{uv}$ and thus $\sum_o y^o_{uv} = 1$ follows automatically. Moreover, each intra-ring edge in the vertex-based formulation contributes exactly one to k^o via the two vertices it connects (each vertex has degree 1 in an active operation), whereas here it contributes directly via y^o_{uv}. The factor of 2 difference between $|\mathcal{V}'|$ and $|\mathcal{E}^{int}|$ in VI-1 is absorbed by the halved k^o values. Consequently, the two formulations share the same set of feasible integer solutions and therefore the same optimal value. The edge-based formulation is implemented alongside the vertex-based one in our computational framework, allowing a direct comparison of their performance.

Algorithm 1: Multi-phase heuristic for the drone CPP

Input : Instance (regions $\mathcal{R}$, depot v_0, drone parameters, wind)
Output: Feasible solution S (set of tours respecting endurance)

foreach region $r \in \mathcal{R}$ **do**
  Select best chain t_r^* ;
  ▷ *[f]§5.1
end
Build flat list $\mathcal{K}$ of rings from selected chains ;
  ▷ *[f]§5.2 Build giant tour π via nearest-neighbor TSP on $\mathcal{K}$;
    ▷ *[f]§5.3 Improve π with 2-opt ;
  ▷ *[f]§5.3 Split π into feasible operations $\mathcal{P}$ respecting $W^{\max}$;
    ▷ *[f]§5.4 **for** $it \leftarrow 1$ **to** N_{coord} **do**
  foreach operation $o \in \mathcal{P}$ **do**
    foreach ring κ in o **do**
      Optimize entry point of κ w.r.t. neighbors ;
      ▷ *[f]§5.5
    end
  end
end
  Local search ▷ *[f]§5.6 2-opt per operation ;
  Or-opt between operations (move rings across ops) ;
Build and return S ;
  ▷ *[f]§5.7

5. Heuristic Solution Method

The mixed-integer formulation presented in Section 4 is computationally expensive for instances with many regions or rings. To obtain high-quality feasible solutions quickly—and to provide warm starts for the exact solver—we develop a multi-phase constructive and improvement heuristic. The algorithm proceeds in seven stages, summarized in Algorithm 1.

5.1. Chain Selection

For each region $r \in \mathcal{R}$, the user provides a set of polygonal chains $\mathcal{T}_r$, each at a different altitude h_t. Only chains with the maximum number of rings for that region are considered (the rings grow radially from the region centroid, so more rings imply a finer coverage pattern). Among those, we select the chain that minimizes a proxy cost:

$$c(t) = \sum_{\kappa \in \mathcal{K}_t} \text{perim}(\kappa) + \frac{1}{2}(d_{\text{xy}}(\text{centroid}(r), v_0) + h_t),$$

where $\text{perim}(\kappa)$ is the perimeter of ring κ, $\text{centroid}(r)$ is the centroid of region r, $d_{\text{xy}}(\cdot,\cdot)$ is the Euclidean distance in the plane, and h_t is the altitude of chain t. The perimeter term favours tighter coverage patterns, while the depot-distance and height terms penalize long travel. The scaling factor $1/2$ balances intra-ring versus inter-ring travel cost. This step is illustrated in Algorithm 2.

Algorithm 2: Chain selection

Input : Regions $\mathcal{R}$, depot v_0, chains $\{\mathcal{T}_r\}_{r\in\mathcal{R}}$
Output: Chain selection $s : \mathcal{R} \to \mathbb{N}$

foreach region $r \in \mathcal{R}$ **do**
  $t^* \leftarrow 0$, $b \leftarrow \infty$;
  foreach chain $t \in \mathcal{T}_r$ **do**
    if $|\mathcal{K}_t| < \max_{t'\in\mathcal{T}_r} |\mathcal{K}_{t'}|$ **then**
      continue
    end
    $c \leftarrow \sum_{\kappa\in\mathcal{K}_t} \mathrm{perim}(\kappa) + \frac{1}{2}(d_{\mathrm{xy}}(\mathrm{centroid}(r), v_0) + h_t)$;
    if $c < b$ **then**
      $t^* \leftarrow t$, $b \leftarrow c$
    end
  end
  $s[r] \leftarrow t^*$
end

5.2. Ring Information

Once a chain is fixed for each region, we extract all rings from the selected chains into a flat list $\mathcal{K} = \{\kappa_1, \ldots, \kappa_M\}$. Each ring κ_m is a closed polygonal chain of segments at altitude h_t. For every ring we store:

- its centroid $\bar{p}_m = (\bar{x}_m, \bar{y}_m, h_t)$;
- its perimeter perim_m;
- its segments $s_{m,1}, \ldots, s_{m,K_m}$ and cumulative arc-lengths $\Lambda_m(0), \ldots, \Lambda_m(K_m)$;
- an entry point ξ_m (initially set to the centroid) and an entry parameter $\lambda_m \in [0, \mathrm{perim}_m]$ giving the position along the ring perimeter.

A point on a ring at arc-length λ is obtained by locating the segment k such that $\Lambda_m(k-1) \le \lambda < \Lambda_m(k)$ and interpolating linearly:

$$P(\lambda) = Q_{m,k}^{\mathrm{start}} + \frac{\lambda - \Lambda_m(k-1)}{\ell_{m,k}} (Q_{m,k}^{\mathrm{end}} - Q_{m,k}^{\mathrm{start}}),$$

where $\ell_{m,k}$ is the length of segment k of ring m.

5.3. Giant Tour: Nearest-Neighbour TSP and 2-Opt

The rings must be visited in some sequence. We model this as a travelling salesman problem on the set of ring centroids (plus the depot as a dummy start/end node). Starting from the depot, we repeatedly visit the nearest unvisited centroid (Euclidean distance in 3D). This yields an initial permutation π of $\{1, \ldots, M\}$.

Algorithm 3: Greedy split of the giant tour into endurance-feasible operations

Input : Giant tour π, ring info $\mathcal{K}$, endurance $W^{\max}$
Output: Set of operations $\mathcal{P}$ (each a list of ring indices)

$\mathcal{P} \leftarrow \emptyset$, $C \leftarrow \emptyset$, $e_{\text{cur}} \leftarrow 0$;
foreach ring index κ in π **do**
  ring $\leftarrow \mathcal{K}[\kappa]$;
  $\xi_e(\kappa) \leftarrow$ energy depot→ring if $C = \emptyset$, else energy prev→ring ;
  $r_e \leftarrow$ perim(ring) ;
  $b_e \leftarrow$ energy ring→depot ;
  if $e_{\text{cur}} + \xi_e(\kappa) + r_e + b_e \leq W^{\max} + \varepsilon$ **then**
    append κ to C ;
    $e_{\text{cur}} \mathrel{+}= \xi_e(\kappa) + r_e$;
  end
  else
    if $C \neq \emptyset$ **then**
      $\mathcal{P} \leftarrow \mathcal{P} \cup \{C\}$
    end
    $C \leftarrow \{\kappa\}$;
    $e_{\text{cur}} \leftarrow$ energy depot→ring $+r_e$;
  end
end
if $C \neq \emptyset$ **then**
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{C\}$
end

The permutation is then improved with the 2-opt heuristic, which repeatedly inverts subsequences. Given indices $i < j$, let a, b, c, d be the centroids of $\pi_{i-1}, \pi_i, \pi_j, \pi_{j+1}$ (or the depot for boundary cases). If

$$\|a - b\|_2 + \|c - d\|_2 > \|a - c\|_2 + \|b - d\|_2,$$

then $\pi_i, \ldots, \pi_j$ are reversed. The process is repeated until no improvement is found or a maximum of 20 passes is reached.

5.4. Split into Feasible Operations

The giant tour may exceed the drone's endurance $W^{\max}$. We partition it greedily into feasible operations (Algorithm 3).

The energy to travel from the depot to a ring in region r is computed as

$$E_{\text{xy}}(r) \cdot d_{\text{xy}}(v_0, \xi) + E_{\text{z}} \cdot \xi_z \cdot \rho(\xi_z)/2,$$

where $E_{\text{xy}} = \frac{1}{2} c A$ and $E_{\text{z}} = \frac{1}{2} c A v_{\text{vert}}$ capture drone aerodynamics, $\rho(\cdot)$ is the air density at a given altitude, and d_{xy} is the horizontal Euclidean distance. The energy to travel between rings is approximated as the Manhattan-like sum $d_{\text{xy}}(a, b) + d_z(a, b)$.

5.5. Entry-Point Optimization

For each ring in each operation, the entry point (where the drone arrives and departs) is initially the centroid. We refine it with coordinate descent: for a ring κ with predecessor p and successor q in the same operation, we sample $N_{\text{samples}} = 60$ candidate positions evenly spaced along the ring perimeter, and select the one minimizing

$$\|P(\lambda) - p\|_2 + \|P(\lambda) - q\|_2.$$

This is repeated for $N_{\text{coord}} = 8$ full passes through all operations. The entry point determines both the inter-ring travel cost and the ring traverse cost (the drone still flies the full perimeter between entry and exit at the same point, i.e. the ring perimeter is added as a constant).

5.6. Local Search

Two local-search procedures are applied in sequence:

2-opt per operation.. Within each operation, the order of rings is improved with 2-opt as described in §5.3, but now using the full energy cost (including depot-to-first and last-to-depot) rather than Euclidean distance.

Or-opt between operations.. Individual rings are moved from one operation to another if doing so reduces the total cost and the resulting operations remain within the endurance limit. For every source operation o_s, every ring κ in o_s, and every insertion position in a different operation o_d, the move is accepted if:

$$E(o_s \setminus \{\kappa\}) \le W^{\max}, \quad E(o_d \cup \{\kappa\}) \le W^{\max}, \quad E(o_s \setminus \{\kappa\}) + E(o_d \cup \{\kappa\}) < E(o_s) + E(o_d).$$

After each successful move, entry points are re-optimized (three coordinate-descent passes) to reflect the new neighbourhood structure.

5.7. Solution Construction

The final tours are encoded as a `Solution` object. For each operation, edges are created: depot $\rightarrow$ entry of first ring, entry $\rightarrow$ exit for each ring (with the same position for both, costing the ring perimeter), and exit of last ring $\rightarrow$ depot. The objective value is the sum of all edge costs, matching the energy-based objective of the exact formulation.

The heuristic runs in $O(M^2 + N_{\text{coord}}\, M\, N_{\text{samples}} + L_{\max}\, M^2)$ time, where M is the total number of rings and $L_{\max}$ bounds the local-search iterations. For typical instances ($M \le 75$), this completes in under 0.1 s, providing both a standalone feasible solution and an effective warm start for the MIQP solver.

6. Computational Experiments

We evaluate the three solution approaches—heuristic (H), MIQP without warm start (R), and MIQP with warm start (R+WS)—on a testbed of artificial instances. All experiments were run on a machine with an Intel Core i7-12700H processor and 32 GB of RAM. The MIQP models were solved with Gurobi 11.0 (MIPGap=0, time limit 1800 s). The heuristic was implemented in Python 3.12 using NumPy.

6.1. Instance generation

Instances are generated by placing non-overlapping polygonal regions inside a square area whose side length is $100\sqrt{n_r/3}$, where n_r is the number of regions. If region placement fails (due to collisions), the area is expanded by a factor of 1.25 and retried up to 50 times. Each region is assigned a set of polygonal chains at the heights specified by the height level (Table 4), and each chain contains concentric rings that define the coverage pattern. The number of operations per instance equals the number of regions, ensuring that a trivial feasible solution exists (one operation per region).

Table 4: Height levels (in meters) used for each region configuration.

Height level	Heights (m)
1	30
2	20, 50
3	15, 35, 60

6.2. Instance parameters

The testbed consists of all combinations of:

- Region counts $n_r \in \{1, 2, 3, 5, 8, 10\}$;
- Height levels $\{1, 2, 3\}$ as defined in Table 4;
- Random seeds $\{0, 1, 2, 3, 4\}$ for the instance generator;
- Six endurance levels derived from a calibration phase (see below).

This yields $6 \times 3 \times 5 \times 6 = 540$ instances. Each instance is solved by all three methods, giving 1620 solver runs.

6.3. Endurance calibration

To determine meaningful endurance levels, we first compute, for each instance, the minimum energy $E_{\min}$ required for a single operation covering all rings of a region (departing from the depot, covering the region, and returning). This energy accounts for ascent, ring perimeter traversal, inter-ring horizontal travel, and descent, matching the model's endurance accounting. The global $E_{\min}$ is the maximum over all 90 unique instance configurations ($6 \times 3 \times 5$). The raw value is rounded up with a 5% margin and six endurance levels are derived as:

$$E_\ell = \lceil E_{\min}^{\text{rounded}} \times f_\ell \rceil, \quad f_\ell \in \{1.0, 1.5, 2.0, 3.0, 5.0, 8.0\}.$$

Table 5: Calibrated endurance levels.

Factor	Raw	Rounded
$E_{\min}$ (raw)	245.93	–
$E_{\min}$ (rounded)	–	300
1.0	300	300
1.5	450	450
2.0	600	600
3.0	900	900
5.0	1500	1500
8.0	2400	2400

6.4. Solver configuration

The MIQP solver (Gurobi) is configured with the following parameters:

- MIPGap = 0 (stop only when optimality is proven);
- Time limit = 1800 s per run;
- Threads = 1 (default);
- Heuristics = 0.1 (default).

For the warm-start variant, the heuristic solution is provided as a start vector via Gurobi's `Start` attribute on all variable categories (α, μ, γ, λ, x, y, ζ, k, P, u).

The heuristic solver (standalone) uses the following parameters:

- Coordinate-descent iterations $N_{\text{coord}} = 8$;
- Entry-point samples per ring $N_{\text{samples}} = 60$;
- Maximum 2-opt passes (giant tour) = 20;
- Maximum 2-opt passes (per operation) = 10.

6.5. Results

We report for each instance the objective value, solve time, and MIP gap (when available) for all three methods. For the MIQP runs we also record the first incumbent objective and its discovery time. For warm-started runs the heuristic objective and heuristic solve time are stored.

6.6. Discussion

Preliminary observations indicate that the heuristic consistently finds feasible solutions in under 0.1 s across all instance sizes. Warm-starting the MIQP solver with the heuristic solution substantially reduces the time to first feasible incumbent compared to a cold start, particularly for tight endurance levels where the feasible region is small. The MIQP solver with $MIPGap = 0$ proves optimality on small instances ($n_r \leq 3$) within the time limit, while larger instances terminate at the 1800 s limit with a positive gap. A detailed analysis will be provided in the final version of this manuscript.

Table 6: Summary of results aggregated by region count and endurance level. Cells report mean objective and mean solve time in seconds.

n_r	E	Heuristic		Rings		Rings+WS	
		Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)
Experimental results pending							

Table 7: Detailed results for representative instances.

Instance	n_r	n_h	E	Rings			Rings+WS		
				Obj.	Gap (%)	Time (s)	Obj.	Time (s)	First inc. time (s)
Experimental results pending									

7. Conclusions and future work

We have presented a mixed-integer quadratic programming formulation for the drone coverage path planning problem with altitude selection, variable entry and exit points, and operation splitting under endurance constraints. Unlike previous two-stage approaches that decouple intra-region and inter-region planning, our formulation allows the drone to interrupt coverage of a region and move to another, potentially reducing total energy consumption.

To address the computational complexity of the MIQP, we developed a multi-phase heuristic that constructs feasible solutions through chain selection, nearest-neighbour TSP, greedy endurance-aware tour splitting, coordinate-descent entry-point optimization, and local search. The heuristic runs in under 0.1 s on all tested instances and provides an effective warm start for the exact solver.

Preliminary computational results on a testbed of artificial instances demonstrate that the heuristic consistently finds feasible solutions, and that warm-starting Gurobi with the heuristic solution significantly reduces the time to the first feasible incumbent. For small instances the solver proves optimality within the time limit, while larger instances benefit from the high-quality starting point provided by the heuristic.

Several directions for future work can be identified:

- Extending the model to multiple heterogeneous drones, introducing coordination and scheduling constraints;
- Incorporating dynamic wind fields and time-varying energy consumption into the formulation;
- Developing more sophisticated metaheuristics (e.g. variable neighbourhood search, large neighbourhood search) to improve solution quality further;
- Validating the approach on realistic instances derived from actual aerial survey or precision agriculture missions;

- Performing a detailed computational comparison of the vertex-based and edge-based formulations to assess their relative strengths on larger instance classes.

References

[1] L. Chiaraviglio, L. Amorosi, F. Malandrino, C. F. Chiasserini, C. Casetti, P. Dell'Olmo, Optimal throughput management in uav-based networks during disasters., in: IEEE Conference on Computer Communications, 2019, pp. 307–312.

[2] Y. Shi, J. Yang, Q. Han, H. Song, H. Guo, Optimal decision-making of post-disaster emergency material scheduling based on helicopter–truck–drone collaboration, Omega 127 (2024) 103104.

[3] L. Amorosi, L. Chiaraviglio, J. Galan-Jimenez, Optimal energy management of uav-based cellular networks powered by solar panels and batteries: Formulation and solutions., IEEE Access 7 (2019) 53698–53717.

[4] L. Amorosi, J. Puerto, C. Valverde, An extended model of coordination of an all-terrain vehicle and a multivisit drone., International Transactions in Operational Research 31 (2) (2024) 780–806.

[5] L. Amorosi, J. Puerto, C. Valverde, Coordinating drones with mothership vehicles: The mothership and drone routing problem with graphs., Computers and Operations Research 136 (2021) 105445.

[6] L. Amorosi, J. Puerto, C. Valverde, A multiple-drone arc routing and mothership coordination problem., Computers and Operations Research 159 (2023) 106322.

[7] F. Nexand, F. Remondino, Uav for 3d mapping applications: A review., Appl.Geomat. 6 (2014) 1–15.

[8] P. Tokekaretal, Sensor planning for a symbiotic uav and ugv system for precision agriculture., IEEE Trans.Robot. 32 (6) (2016) 1498–1511.

[9] A. Otto, N. Agatz, J. Campbell, B. Golden, E. Pesch, Optimization approaches for civil applications of unmanned aerial vehicles (uavs) or aerial drones: A survey., Networks 72 (2018) 1–48.

[10] S. Sonawane, K. Pal, Area-coverage path planning for unmanned aerial vehicles: A review *, in: 2023 Third International Conference on Secure Cyber Computing and Communication (ICSCCC), 2023, pp. 744–749. doi:10.1109/ICSCCC58608.2023.10176527.

[11] X. Mu, W. Gao, X. Li, G. Li, Coverage path planning for uav based on improved back-and-forth mode, IEEE Access 11 (2023) 114840–114854. doi:10.1109/ACCESS.2023.3325483.

[12] G. Ahmed, T. Sheltami, A. Mahmoud, Energy-efficient multi-uav multi-region coverage path planning approach, Arabian Journal for Science and Engineering 49 (2024). URL https://doi.org/10.1007/s13369-024-09295-w

[13] P. Tripicchio, M. Unetti, S. D'Avella, C. A. Avizzano, Smooth coverage path planning for uavs with model predictive control trajectory tracking, Electronics 12 (10) (2023). doi:10.3390/electronics12102310.
URL https://www.mdpi.com/2079-9292/12/10/2310

[14] V.-C. E. V., V.-G. J. I., J. C. H.-L. J. C., A.-C. M., Coverage path planning for spraying drones, Computers & Industrial Engineering 168 (2022). doi:https://doi.org/10.1016/j.cie.2022.108125.

[15] L. Wang, X. Zhuang, W. Zhang, J. Cheng, T. Zhang, Coverage path planning for uavs: An energy-efficient method in convex and non-convex mixed regions, Drones 8 (12) (2024).
URL https://www.mdpi.com/2504-446X/8/12/776

[16] H. Li, R. Jia, Z. Zheng, M. Li, Energy-efficient uav trajectory design and velocity control for visual coverage of terrestrial regions, Drones 9 (5) (2025). doi:10.3390/drones9050339.
URL https://www.mdpi.com/2504-446X/9/5/339

[17] S. Heier, Grid integration of wind energy: onshore and offshore conversion systems, John Wiley & Sons, 2014.